NovaTech Solutions Pvt. Ltd.

NovaDesk Developer Guide

Document ID: TECH-DEV-001

Version: 3.0

Product: NovaDesk Enterprise Service Management Platform

Prepared By: Platform Engineering Team

Effective Date: January 2026

Classification: Internal Use Only

Document Control

Field	Value
Document Name	NovaDesk Developer Guide
Product	NovaDesk
Version	3.0
Owner	Platform Engineering
Review Cycle	Every Major Release
Last Updated	January 2026

Table of Contents

1. Introduction

The NovaDesk Developer Guide provides engineering teams with the standards, tools, and procedures required to develop, maintain, test, and deploy the NovaDesk Enterprise Service Management Platform.

This document is intended for:

- Software Engineers
- QA Engineers
- DevOps Engineers
- Technical Leads
- Platform Engineers
- New Engineering Team Members

It serves as the primary engineering reference for local development, project structure, coding practices, and build processes.

Product usage instructions are documented separately in the **NovaDesk User Guide (PROD-ND-001)**.

---

2. Development Philosophy

NovaDesk follows modern software engineering principles to ensure maintainability, scalability, reliability, and security.

Development teams should prioritize:

- Simplicity
- Readability
- Testability
- Maintainability
- Automation
- Security by Design
- Continuous Improvement

Engineering Principles

Modular Design

Every component should have a clearly defined responsibility.

Business logic, presentation logic, persistence, and infrastructure code should remain independent wherever practical.

Clean Code

Developers should write code that is:

- Readable
- Predictable
- Well documented
- Easy to test
- Easy to refactor

Complex implementations should include explanatory comments where necessary.

Reusability

Reusable components should be preferred over duplicated implementations.

Shared functionality should be placed in common libraries or utility modules.

Fail Fast

Applications should detect invalid input early and return meaningful error messages rather than allowing failures to propagate unpredictably.

Security First

Security considerations must be incorporated during design rather than added later in the development lifecycle.

Every feature should be evaluated for potential security risks before implementation.

---

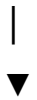
3. System Architecture

NovaDesk follows a layered architecture that separates presentation, business logic, and persistence.

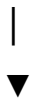
Browser

Web Application

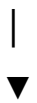
REST API



Business Services



Repositories



Database

This separation improves scalability and simplifies maintenance.

High-Level Components

The platform consists of the following major components:

- Web Frontend
- REST API
- Authentication Service
- Ticket Service
- Asset Service
- Knowledge Base Service
- Notification Service
- Reporting Service
- Workflow Engine
- Database Layer

Each component communicates through well-defined interfaces.

Request Lifecycle

A typical request follows these stages:

1. Client sends HTTP request.
2. Authentication middleware validates credentials.
3. Authorization middleware verifies permissions.
4. Controller validates request.
5. Business service executes logic.
6. Repository accesses database.
7. Response is returned as JSON.

Every request receives a unique Correlation ID for traceability.

---

4. Technology Stack

NovaDesk is built using proven enterprise technologies.

Backend

Component	Technology
Language	Java 21
Framework	Spring Boot
Build Tool	Maven
ORM	Hibernate
API	Spring MVC

Frontend

Component	Technology
Framework	React
Language	TypeScript
Styling	Tailwind CSS
State Management	Redux Toolkit

Database

Component	Technology
Primary Database	PostgreSQL 15
Cache	Redis
Search	Elasticsearch

Infrastructure

Component	Technology
Containerization	Docker
Orchestration	Kubernetes
CI/CD	GitHub Actions
Monitoring	Prometheus
Dashboards	Grafana
Logging	ELK Stack

Development Tools

Recommended software versions:

Tool	Version
Java	21 LTS
Maven	3.9+
Node.js	20 LTS
Docker	27.x
Git	2.45+

---

5. Repository Structure

NovaDesk follows a standardized repository layout.

novadesk/

├── backend/

├── frontend/
├── database/
├── infrastructure/
├── docs/
├── scripts/
├── tests/
├── docker/
├── .github/
├── README.md
├── docker-compose.yml
└── LICENSE

Backend Structure

backend/

src/

main/

java/

config/

controller/

service/

repository/

entity/

dto/

security/

exception/

util/

Responsibilities:

- **config** – Application configuration
- **controller** – REST controllers
- **service** – Business logic
- **repository** – Database access
- **entity** – Database entities
- **dto** – Request and response models
- **security** – Authentication and authorization
- **exception** – Global exception handling
- **util** – Shared utilities

Frontend Structure

frontend/

src/

components/

pages/

hooks/

services/

store/

assets/

styles/

This organization separates reusable UI components from application pages and business logic.

---

6. Development Environment

Developers should use supported operating systems.

Operating System	Supported
Windows 11	Yes
Ubuntu 22.04 LTS	Yes
macOS Sonoma	Yes

Required Software

Install the following before beginning development:

- Java Development Kit (JDK) 21
- Maven 3.9 or later
- Node.js 20 LTS
- Docker Desktop
- Git
- PostgreSQL Client
- Redis Client

Recommended IDEs

Supported Integrated Development Environments:

- IntelliJ IDEA Ultimate
- Visual Studio Code

- Eclipse IDE

Developers should install extensions for:

- Java
- Spring Boot
- Docker
- YAML
- Git
- EditorConfig

Local Services

A standard local environment includes:

- PostgreSQL
- Redis
- Elasticsearch

These services may be started using Docker Compose.

---

7. Local Installation

Clone the repository.

git clone https://github.com/novatechsolutions/novadesk.git

Move into the project directory.

cd novadesk

Start supporting infrastructure.

docker compose up -d

Build the backend.

cd backend

mvn clean install

Run the backend service.

mvn spring-boot:run

Start the frontend.

cd ../frontend

npm install

npm run dev

Verify the application.

Backend

http://localhost:8080

Frontend

http://localhost:3000

---

8. Environment Variables

NovaDesk uses environment variables for runtime configuration.

Example:

APP_ENV=development

SERVER_PORT=8080

DATABASE_URL=jdbc:postgresql://localhost:5432/novadesk

DATABASE_USERNAME=novadesk

DATABASE_PASSWORD=password

REDIS_HOST=localhost

REDIS_PORT=6379

JWT_SECRET=replace-with-secure-secret

LOG_LEVEL=INFO

Sensitive credentials should never be committed to version control.

Developers should use local .env files or approved secret management systems.

---

9. Build Process

NovaDesk uses Maven for backend builds and npm for frontend builds.

Backend Build

mvn clean package

This process performs:

- Dependency resolution
- Source compilation
- Unit testing
- Packaging
- Artifact generation

Frontend Build

npm run build

The production build generates optimized static assets suitable for deployment.

Build Artifacts

Generated artifacts include:

- Backend executable JAR
- Frontend production assets
- Build reports
- Test reports
- Coverage reports

Artifacts should be archived by the CI/CD pipeline for traceability.

Revision History

Version	Date	Description
1.0	January 2024	Initial Developer Guide
2.0	August 2025	Updated Repository Structure
3.0	January 2026	Added Architecture, Environment Setup, and Build Process

---

10. Coding Standards

All contributors must follow the NovaTech Engineering Coding Standards to ensure consistency, maintainability, and readability across the codebase.

General Principles

Developers should:

- Write clear, self-explanatory code.
- Keep methods focused on a single responsibility.
- Prefer composition over inheritance where appropriate.
- Avoid duplicate logic.
- Minimize deeply nested control structures.
- Document non-obvious implementation decisions.

Naming Conventions

Element	Convention	Example
Class	PascalCase	TicketService
Interface	PascalCase	NotificationProvider
Method	camelCase	createTicket()
Variable	camelCase	ticketCount
Constant	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
Package	lowercase	com.novatech.service
React Component	PascalCase	DashboardWidget

Meaningful names should be preferred over abbreviations.

Code Formatting

Formatting standards:

- Indentation: **4 spaces** (Java), **2 spaces** (TypeScript).
- Maximum line length: **120 characters**.
- UTF-8 file encoding.
- Unix (LF) line endings.
- Remove trailing whitespace before committing.

All source files should be automatically formatted using the project's configured formatter.

Comments

Comments should explain **why**, not **what**.

Appropriate uses include:

- Complex algorithms
- Business rules
- Security considerations
- Performance optimizations

Avoid redundant comments that simply restate the code.

---

11. Git Workflow

NovaDesk uses **Git** for version control with a protected branching model.

All code changes must be committed through feature branches and reviewed before merging.

Branch Types

Branch	Purpose
main	Production-ready code
develop	Integration branch
feature/*	New feature development
bugfix/*	Bug fixes
hotfix/*	Production fixes

release/* Release preparation

Direct commits to main are prohibited.

Feature Development Workflow

1. Create a branch from develop.
2. Implement the feature.
3. Write unit tests.
4. Run local validation.
5. Push the branch.
6. Open a Pull Request.
7. Complete code review.
8. Merge after approval.

Commit Message Convention

Commit messages should follow this format:

type(scope): concise description

Examples:

feat(ticket): add SLA breach notification

fix(search): resolve duplicate search results

docs(api): update authentication examples

refactor(asset): simplify repository implementation

Supported commit types:

- feat
- fix
- docs
- refactor
- test

- chore
- perf
- ci

---

12. Pull Request Guidelines

Every Pull Request (PR) should include:

- Summary of changes
- Related issue or task ID
- Testing performed
- Screenshots (if UI changes)
- Deployment impact
- Rollback considerations

Review Requirements

A Pull Request requires:

- At least **two approvals** from engineering reviewers.
- Successful CI pipeline execution.
- No unresolved review comments.
- No critical security findings.

The author must not merge their own Pull Request without the required approvals.

---

13. Testing Guidelines

Testing is mandatory for all production code.

Test Pyramid

NovaDesk follows a layered testing strategy:

1. Unit Tests
2. Integration Tests
3. End-to-End Tests

Unit tests should provide the majority of test coverage.

Unit Testing

Unit tests should:

- Test business logic independently.
- Execute quickly.
- Avoid external dependencies.
- Use descriptive test names.

Target code coverage:

Minimum 80% for business service classes.

Integration Testing

Integration tests verify interactions between:

- REST controllers
- Services
- Repositories
- Database
- External integrations

Integration tests should execute against isolated test environments.

End-to-End Testing

End-to-end tests validate complete user workflows, including:

- Login
- Ticket creation
- Ticket assignment
- Asset allocation
- Knowledge article publication

Critical business workflows should be executed before every production release.

---

14. Logging Standards

Application logs support troubleshooting, monitoring, and auditing.

Log Levels

Level	Usage
TRACE	Detailed diagnostic information
DEBUG	Development troubleshooting
INFO	Normal application events
WARN	Recoverable issues
ERROR	Failures requiring investigation

Sensitive information such as passwords, access tokens, or personally identifiable information (PII) must never be written to logs.

Structured Logging

Every log entry should include:

- Timestamp
- Log Level
- Correlation ID
- Service Name
- User ID (when available)
- Request ID

Structured logs improve searchability and observability.

---

15. Exception Handling

Exceptions should be handled consistently across the platform.

Best Practices

Developers should:

- Catch only exceptions they can handle.
- Avoid swallowing exceptions silently.
- Return meaningful error messages.
- Log unexpected failures.

- Preserve stack traces where appropriate.

Global exception handlers should convert internal exceptions into standardized API responses.

Validation Errors

Validation failures should return:

- HTTP **400 Bad Request**
- Clear field-level validation messages

Business validation errors should use standardized application error codes.

---

16. Performance Guidelines

Performance should be considered throughout development.

Database Optimization

Developers should:

- Use indexes appropriately.
- Avoid unnecessary database queries.
- Minimize N+1 query problems.
- Use pagination for large datasets.
- Batch updates where practical.

API Performance

Recommended practices:

- Compress responses where supported.
- Cache frequently accessed data.
- Limit payload sizes.
- Avoid unnecessary synchronous processing.
- Use asynchronous operations for long-running tasks.

Frontend Performance

Frontend developers should:

- Lazy-load large modules.
- Minimize bundle size.
- Optimize images.
- Avoid unnecessary re-renders.
- Use efficient state management.

---

17. Secure Development Guidelines

Security must be incorporated into every phase of development.

Input Validation

All external input should be validated before processing.

Validation should include:

- Required fields
- Length limits
- Data types
- Allowed values
- Business rules

Authentication

Protected endpoints must verify:

- User identity
- Token validity
- Session status

Authentication logic should never be bypassed.

Authorization

Authorization checks must occur on the server side for every protected operation.

Client-side validation alone is not sufficient.

Sensitive Data

Developers must never:

- Hardcode credentials.
- Store passwords in plain text.
- Commit secrets to source control.
- Log authentication tokens.
- Expose internal system information in error messages.

Secrets should be managed using approved secret management solutions.

Dependency Management

Third-party libraries should:

- Come from trusted sources.
- Be actively maintained.
- Be scanned regularly for known vulnerabilities.
- Be updated according to the organization's patch management schedule.

---

18. Documentation Standards

Every new feature should include updated documentation.

Documentation should cover:

- Feature overview
- Configuration changes
- API modifications
- Database changes
- User impact
- Deployment considerations

Technical documentation should be reviewed alongside code changes.

19. Continuous Integration and Continuous Deployment (CI/CD)

NovaDesk uses an automated CI/CD pipeline to validate, build, test, and deploy software changes.

Every code change passes through automated quality gates before deployment.

Pipeline Stages

The standard pipeline consists of:

1. Source Checkout
2. Dependency Installation
3. Static Code Analysis
4. Unit Testing
5. Integration Testing
6. Build Packaging
7. Security Scanning
8. Container Image Creation
9. Deployment
10. Post-Deployment Validation

A deployment proceeds only if all mandatory stages complete successfully.

GitHub Actions Workflow

NovaDesk uses **GitHub Actions** for automation.

Typical workflow events include:

- Pull Request Opened
- Pull Request Updated
- Merge to develop
- Merge to main
- Release Tag Creation

Separate workflows exist for backend, frontend, infrastructure, and documentation.

Quality Gates

The CI pipeline enforces:

- Successful compilation
- Passing unit tests
- Passing integration tests
- Static analysis with no critical issues

- Dependency vulnerability scan
- Minimum code coverage requirements

Builds failing any mandatory quality gate cannot be promoted.

---

20. Docker Development

Docker provides a consistent runtime environment across developer workstations.

Local Containers

A standard development environment includes:

- PostgreSQL
- Redis
- Elasticsearch
- Backend Service
- Frontend Development Server

Developers should use Docker Compose to start required services.

Container Build

Backend image:

docker build -t novadesk-backend .

Frontend image:

docker build -t novadesk-frontend .

Docker Compose

Start all services:

docker compose up -d

Stop all services:

docker compose down

Rebuild services after dependency changes:

docker compose up --build

---

21. Deployment Process

Production deployments follow a controlled release process.

Deployment Environments

Environment	Purpose
Development	Feature development
Integration	Shared testing
Quality Assurance	Functional verification
Staging	Production validation
Production	Customer environment

Code must progress through each environment before production deployment.

Deployment Checklist

Before deployment:

- All automated tests pass.
- Release notes are completed.
- Database migrations are reviewed.
- Rollback plan is documented.
- Stakeholders are notified.
- Monitoring dashboards are prepared.

Rollback Strategy

If a deployment introduces a critical issue:

1. Pause new deployments.
2. Restore the previous application version.
3. Verify application health.
4. Review logs and metrics.
5. Conduct root cause analysis.

Rollback procedures should be tested during release rehearsals.

---

22. Database Migration Guidelines

Schema changes must be version-controlled and repeatable.

Migration Rules

Developers should:

- Create one migration per logical change.
- Keep migrations idempotent where possible.
- Avoid destructive operations during peak business hours.
- Test migrations in non-production environments.

Database migrations should be reviewed during code review.

Migration Naming

Example:

V20260115__add_ticket_priority_index.sql

Migration filenames should include:

- Version
- Date
- Descriptive change summary

---

23. Monitoring and Observability

Operational visibility is essential for maintaining service reliability.

Metrics

NovaDesk monitors:

- CPU Utilization
- Memory Usage
- Request Throughput
- Response Time
- Error Rate
- Database Connections

- Queue Length
- Cache Hit Ratio

Metrics are collected continuously and visualized using Grafana dashboards.

Health Checks

Every service exposes a health endpoint.

Example:

/actuator/health

Health responses indicate:

- Database connectivity
- Cache availability
- Search service availability
- Disk space
- Application status

Alerting

Alerts are configured for:

- High error rates
- Increased response times
- Service unavailability
- Failed deployments
- Database connection failures
- Low disk space

Critical alerts are routed to the on-call engineering team.

---

24. Troubleshooting

Developers should follow a structured troubleshooting process.

Common Issues

Issue	Possible Cause	Resolution

Application fails to start	Missing environment variable	Verify configuration
Database connection failure	Database unavailable	Confirm database service status
Slow API responses	Inefficient query	Review query execution plan
Build failure	Dependency conflict	Update dependency versions
Authentication errors	Expired token	Reauthenticate and retry

Debugging Recommendations

Developers should:

- Review application logs.
- Verify environment variables.
- Confirm service dependencies.
- Reproduce issues locally.
- Check recent deployments.
- Inspect monitoring dashboards.

---

25. Code Review Checklist

Every Pull Request should be evaluated using the following checklist.

Functional Review

- Feature satisfies requirements.
- Edge cases considered.
- Validation implemented.
- Error handling included.

Code Quality

- Readable implementation.
- No duplicated logic.
- Appropriate naming.
- Consistent formatting.

Security Review

- Authentication enforced.
- Authorization verified.
- Input validated.
- Secrets protected.
- Sensitive data excluded from logs.

Performance Review

- Efficient database queries.
- Appropriate caching.
- Pagination for large datasets.
- No unnecessary API calls.

Testing Review

- Unit tests included.
- Integration tests updated.
- Existing tests continue to pass.

---

26. Operational Best Practices

Engineering teams should:

- Deploy small, incremental changes.
- Monitor production after every deployment.
- Document architectural decisions.
- Review technical debt regularly.
- Remove obsolete code.
- Keep dependencies updated.
- Conduct post-incident reviews.
- Automate repetitive operational tasks.

Continuous improvement is expected across all engineering teams.

---

27. Frequently Asked Questions

Q1. Which CI/CD platform does NovaDesk use?

NovaDesk uses **GitHub Actions** for continuous integration and deployment automation.

Q2. Which environments are maintained?

Development, Integration, Quality Assurance, Staging, and Production.

Q3. What should I do before deploying to production?

Ensure all quality gates pass, review database migrations, prepare rollback procedures, notify stakeholders, and verify monitoring readiness.

Q4. Where can I check service health?

Service health is available through the /actuator/health endpoint.

Q5. What should I do if a production deployment fails?

Pause deployments, execute the rollback plan, verify system health, investigate logs and metrics, and perform a root cause analysis.

Q6. Should database migrations be tested?

Yes. All migrations must be validated in non-production environments before production deployment.

Q7. What monitoring metrics are collected?

CPU utilization, memory usage, request throughput, response time, error rate, database connections, queue length, and cache hit ratio.

Q8. What is required before merging a Pull Request?

Passing automated checks, required approvals, successful testing, and completion of code review requirements.

Q9. Should secrets be committed to source control?

No. Secrets must be stored using approved secret management solutions.

Q10. Who should be notified during critical production incidents?

The on-call engineering team, Technical Lead, Engineering Manager, and other stakeholders according to the incident response process.

Revision History

Version	Date	Description
1.0	January 2024	Initial Developer Guide
2.0	August 2025	Added Development Workflow and Testing Standards
3.0	January 2026	Added CI/CD, Docker, Deployment, Monitoring, and Operational Guidance

---

Related Documents

- NovaDesk User Guide (PROD-ND-001)
- NovaDesk REST API Documentation (TECH-API-001)
- NovaDesk Release Notes (PROD-ND-004)
- Information Security Policy (COMP-SEC-003)
- Employee Handbook (HR-EMP-001)